Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №5

по дисциплине «Логика и основы алгоритмизации в инженерных задачах»

на тему: «Определение характеристик графов»

Выполнили ст.гр. 25ВВВ3:
Волков И.Р.
Итянина В.А.
Приняли:
к.т.н., доцент Юрова О.В.
к.т.н, ст. преподаватель, Деев М.В.

Пенза 2026

Цель работы: изучение характеристик неориентированных графов и разработка программы для их определения по матрице сложности.

Ход работы

Неориентированный граф – граф, в котором рёбра не имеют направления. Если вершины v1-v1 соединены ребром, то связь между ними является взаимной. Поэтому матрица смежности неориентированного графа симметрична относительно главной диагонали.

Размер графа можно определить по количеству его рёбер.

Степень вершины – это количество рёбер, которые с ней соединены, обозначается – deg(vi).

Изолированная вершина – это вершина, у которой вообще нет ребра. В матрице вся строка такой вершины будет состоять из нулей.

Концевая вершина – это вершина, которая соединена только с одной другой вершиной, поэтому deg(vi) = 1.

Доминирующая вершина – это вершина, которая соединена со всеми остальными вершинами графа. Если в графе n вершин, то доминирующая вершина должна иметь степень: deg(vi) = n-1.

Матрица смежности – это квадратная матрица, в которой единица показывает наличие ребра между двумя вершинами, а ноль – его отсутствие.

Задание 1: реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.

Задание 2: реализовать функцию подсчёта числа вхождений заданного элемента в дерево.

Задание 3: изменить функцию добавления элементов для исключения добавления одинаковых символов.

Код:

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

#define N 8
#define MAX_EDGES (N * (N - 1) / 2)
```

```c
int main(void)
{
    int A[N][N];
    int incidence[N][MAX_EDGES];

    int i, j;
    int edges = 0;
    int edge = 0;
    int degree;
    int found;

    if (setlocale(LC_ALL, "Russian_Russia.1251") == NULL)
    {
        if (setlocale(LC_ALL, "ru_RU.UTF-8") == NULL)
        {

        }
    }

    srand((unsigned)time(NULL));


    for (i = 0; i < N; i++)
    {
        for (j = 0; j < N; j++)
        {
            A[i][j] = 0;
        }
    }

    for (i = 0; i < N; i++)
    {
        for (j = i + 1; j < N; j++)
        {
            A[i][j] = rand() % 2;
            A[j][i] = A[i][j];
        }
    }

    printf("МАТРИЦА СМЕЖНОСТИ\n");

    printf("       ");
    for (i = 0; i < N; i++)
    {
        printf("%4d", i + 1);
    }
    printf("\n");

    for (i = 0; i < N; i++)
    {
        printf("v%-4d ", i + 1);
        for (j = 0; j < N; j++)
```

```
    {
        printf("%4d", A[i][j]);
    }
    printf("\n");
}

edges = 0;
for (i = 0; i < N; i++)
{
    for (j = i + 1; j < N; j++)
    {
        if (A[i][j] == 1)
        {
            edges++;
        }
    }
}

printf("\nРазмер графа |E| = %d\n", edges);

printf("\nСтепени вершин:\n");
for (i = 0; i < N; i++)
{
    degree = 0;
    for (j = 0; j < N; j++)
    {
        degree += A[i][j];
    }
    printf("deg(v%d) = %d\n", i + 1, degree);
}

printf("\nИзолированные вершины (степень 0): ");
found = 0;
for (i = 0; i < N; i++)
{
    degree = 0;
    for (j = 0; j < N; j++)
    {
        degree += A[i][j];
    }
    if (degree == 0)
    {
        printf("v%d ", i + 1);
        found = 1;
    }
}
if (found == 0)
{
    printf("нет");
}

printf("\nКонцевые вершины (степень 1): ");
found = 0;
```

```
for (i = 0; i < N; i++)
{
    degree = 0;
    for (j = 0; j < N; j++)
    {
        degree += A[i][j];
    }
    if (degree == 1)
    {
        printf("v%d ", i + 1);
        found = 1;
    }
}
if (found == 0)
{
    printf("нет");
}

printf("\nДоминирующие вершины (степень %d): ", N - 1);
found = 0;
for (i = 0; i < N; i++)
{
    degree = 0;
    for (j = 0; j < N; j++)
    {
        degree += A[i][j];
    }
    if (degree == N - 1)
    {
        printf("v%d ", i + 1);
        found = 1;
    }
}
if (found == 0)
{
    printf("нет");
}
printf("\n");

//инцидентность
for (i = 0; i < N; i++)
{
    for (j = 0; j < MAX_EDGES; j++)
    {
        incidence[i][j] = 0;
    }
}

edge = 0;
for (i = 0; i < N; i++)
{
    for (j = i + 1; j < N; j++)
    {
```

```
            if (A[i][j] == 1)
            {
                if (edge >= MAX_EDGES)
                {
                    printf("Ошибка: превышено MAX_EDGES!\n");
                    return 1;
                }
                incidence[i][edge] = 1;
                incidence[j][edge] = 1;
                edge++;
            }
        }
    }

    printf("МАТРИЦА ИНЦИДЕНТНОСТИ\n");

    printf("       ");
    for (j = 0; j < edges; j++)
    {
        printf("e%-3d", j + 1);
    }
    printf("\n");

    for (i = 0; i < N; i++)
    {
        printf("v%-4d ", i + 1);
        for (j = 0; j < edges; j++)
        {
            printf("%-4d", incidence[i][j]);
        }
        printf("\n");
    }


    printf("\nРазмер графа |E| = %d\n", edges);

    printf("\nСтепени вершин:\n");
    for (i = 0; i < N; i++)
    {
        degree = 0;
        for (j = 0; j < edges; j++)
        {
            degree += incidence[i][j];
        }
        printf("deg(v%d) = %d\n", i + 1, degree);
    }

    printf("\nИзолированные вершины (степень 0): ");
    found = 0;
    for (i = 0; i < N; i++)
    {
        degree = 0;
        for (j = 0; j < edges; j++)
```

```
        {
            degree += incidence[i][j];
        }
        if (degree == 0)
        {
            printf("v%d ", i + 1);
            found = 1;
        }
    }
    if (found == 0)
    {
        printf("нет");
    }

    printf("\nКонцевые вершины (степень 1): ");
    found = 0;
    for (i = 0; i < N; i++)
    {
        degree = 0;
        for (j = 0; j < edges; j++)
        {
            degree += incidence[i][j];
        }
        if (degree == 1)
        {
            printf("v%d ", i + 1);
            found = 1;
        }
    }
    if (found == 0)
    {
        printf("нет");
    }

    printf("\nДоминирующие вершины (степень %d): ", N - 1);
    found = 0;
    for (i = 0; i < N; i++)
    {
        degree = 0;
        for (j = 0; j < edges; j++)
        {
            degree += incidence[i][j];
        }
        if (degree == N - 1)
        {
            printf("v%d ", i + 1);
            found = 1;
        }
    }
    if (found == 0)
    {
        printf("нет");
    }
```

```
    printf("\n");

    return 0;
}
```

Результат работы программы представлен на рисунке 1.

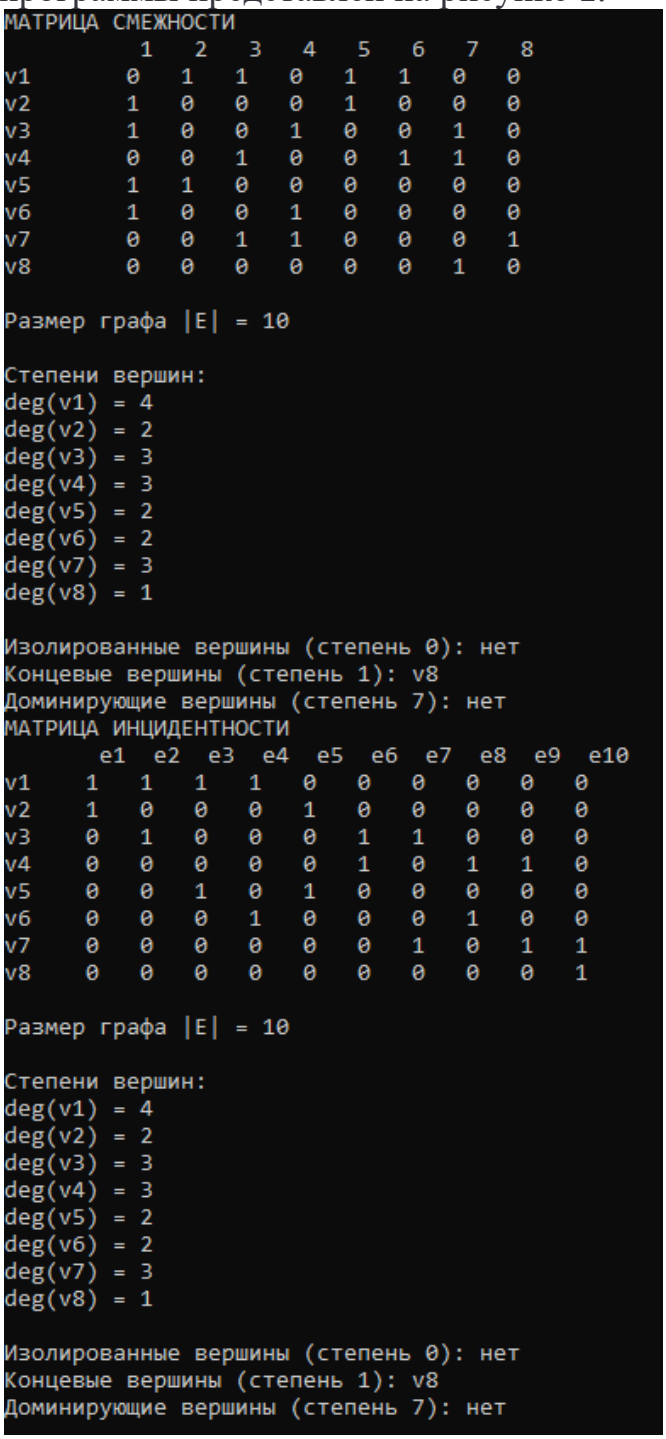


Рис.1 – результат работы программы к заданиям 1-3.

Вывод: в ходе работы была создана программа, которая генерирует случайный неориентированный граф, строит матрицу смежности и вычисляет его характеристики: размер графа, степени вершин, наличие изолированных, концевых и доминирующих вершин.